

Mixture Distribution Project

February 11, 2025

1 Modeling Asset Returns Using Bivariate Mixture Distribution and Monte Carlo Simulation

By Amos Anderson

1.1 Introduction

A mixture distribution, also called a compound distribution, is a univariate or multivariate statistical distribution that “can be expressed as superpositions of component distributions” (Everitt, 2013). Mixture distributions are often used to model scenarios where a population consists of multiple subpopulations, each with distinct statistical characteristics.

In portfolio optimization, some assets within a given portfolio may share common probabilistic characteristics that differ from others in the same portfolio. This results in sub-distributions within the overall portfolio distribution. To realistically model such portfolios, mixture distribution analysis is essential (see Luxenberg et. al, 2022).

1.2 About This Project

In this project, we apply mixture distribution analysis using Python. We construct a bivariate mixture distribution to model the returns of a portfolio of two assets under two distinct market regimes:

1. A favorable market regime, where returns are generally positive.
2. A turbulent market regime, where returns are mostly volatile and possible negative.

We assume that the return of the portfolio under each regime follows a bivariate normal distribution, each with its own mean vector and covariance matrix.

The primary objectives of this project are to:

- Analyze the characteristics of the resulting mixture distribution.
- Compute a theoretical univariate distribution of the portfolio returns.
- Perform a Monte Carlo simulation, generating 500 portfolio return samples from the mixture distribution.
- Compare the simulated empirical distribution with the theoretical univariate distribution derived from the given parameters.

1.3 Problem Development

Consider the context of return of the portfolio of two assets (x_1 and x_2) under two market conditions namely: favorable (Regime 1) and turbulent (Regime 2). The expected returns of the portfolio (made up of assets x_1 and x_2) follows a bivariate normal distribution in each regime. However, the mean and covariance of the portfolio returns depend on which regime is active.

The following are the assumed parameters:

1. **Mixing weights;** $w = \{0.7, 0.3\}$

The above weights implies that:

- Regime 1 occurs with 70% probability.
- Regime 2 occurs with 30% probability.

2. **Mean Vectors;** (μ_1 and μ_2)

These represent the expected returns (mean returns) of the two assets under each regime.

- **Under Regime 1:**

$$\mu_1 = \begin{pmatrix} 0.15 \\ 0.11 \end{pmatrix}$$

- Expected return of asset x_1 is 15%.
- Expected return of asset x_2 is 11%.

- **Under Regime 2:**

$$\mu_2 = \begin{pmatrix} -0.07 \\ -0.02 \end{pmatrix}$$

- Expected return of asset x_1 is -7%.
- Expected return of asset x_2 is -2%.

It is clear why Regime 1 is favorable. It yields positive and high returns. Regime not only yield lower return but actually records negative returns.

3. **Covariance Matrices;** (Σ_1 and Σ_2)

These measure the volatility and correlation of the two assets under each regime.

- **Under Regime 1:**

$$\Sigma_1 = \begin{pmatrix} 0.01 & 0.006 \\ 0.006 & 0.01 \end{pmatrix}$$

- The correlation between the two assets is given by:

$$\rho = \frac{0.006}{\sqrt{(0.01)(0.01)}} = 0.6$$

- Moderate positive correlation between the two assets.
- Both assets have the same volatility of 0.10.

- **Under Regime 2:**

$$\Sigma_2 = \begin{pmatrix} 0.02 & 0.017 \\ 0.017 & 0.02 \end{pmatrix}$$

- The correlation between the two assets is given by:

$$\rho = \frac{0.017}{\sqrt{(0.02)(0.02)}} = 0.85$$

- High positive correlation between the two assets.
- Both assets have equal volatility of 0.1414.

Thus, in our problem, we are looking at Regime 1, which is characterized by high returns, moderate asset correlation, and relatively lower volatility, and Regime 2, which exhibits negative returns, high asset correlation, and high volatility.

1.4 Constructing the Mixture Distribution

We apply python program to construct a mixture distribution based on the given parameters.

```
[139]: # Importing necessary libraries

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import plotly.graph_objects as go
import scipy.stats as stats
from scipy.stats import multivariate_normal
from scipy.stats import norm
```

```
[3]: # Defining the parameters

# Weights
wgts = np.array([0.7,0.3])

# Mean Vectors
mu1 = np.array([0.15,0.11])
mu2 = np.array([-0.07, -0.02])

# Covariance Matrices
cov1 = np.array([[0.01,0.006],[0.006,0.01]])
cov2 = np.array([[0.02,0.017],[0.017,0.02]])
```

```
[5]: # Constructing Mixture distribution

# Defining random variables
x1 = np.linspace(-0.5,0.5,1000) # Asset 1
x2 = np.linspace(-0.35,0.4,1000) # Asset 2
x1, x2 = np.meshgrid(x1,x2)

# Bivariate Normal PDF of Regime 1
regDist1 = wgts[0]*multivariate_normal.pdf(np.dstack([x1,x2]),mean = mu1, cov =  
↪cov1)
```

```

# Bivariate Normal PDF of Regime 2
regDist2 = wgts[1]*multivariate_normal.pdf(np.dstack([x1,x2]),mean = mu2, cov =
    ↪cov2)

# Mixture Distribution
mixDist = np.add(regDist1,regDist2)

```

```

[76]: # We can use the following function achieve the same result, however, we will
    ↪use the above for the mixture distribution

# def mixDistFunc(x1,x2,weights,means,covariances):
#     mixPDF = sum(w*multivariate_normal.pdf(np.dstack([x1,x2]), mean = m, cov
    ↪= c)
#         for w,m,c in zip(weights,means,covariances))
#     return mixPDF

# # Parameters
# portfolio = np.array([0.6, 0.4]) # Portfolio allocation
# weights = wgts # Mixture weights
# means = np.array([mu1, mu2]) # Mean vectors
# covariances = np.array([cov1, cov2]) # Covariance matrices
# # Defining random variables
# x1 = np.linspace(-0.5,0.5,1000) # Asset 1
# x2 = np.linspace(-0.35,0.4,1000) # Asset 2
# x1, x2 = np.meshgrid(x1,x2)

# # Compute the univariate distribution
# mixDistFunc = uniDistFunc(x1, x2, weights, means, covariances)

```

1.5 Visualizing the Mixture Distribution

```

[8]: # Contour Plot

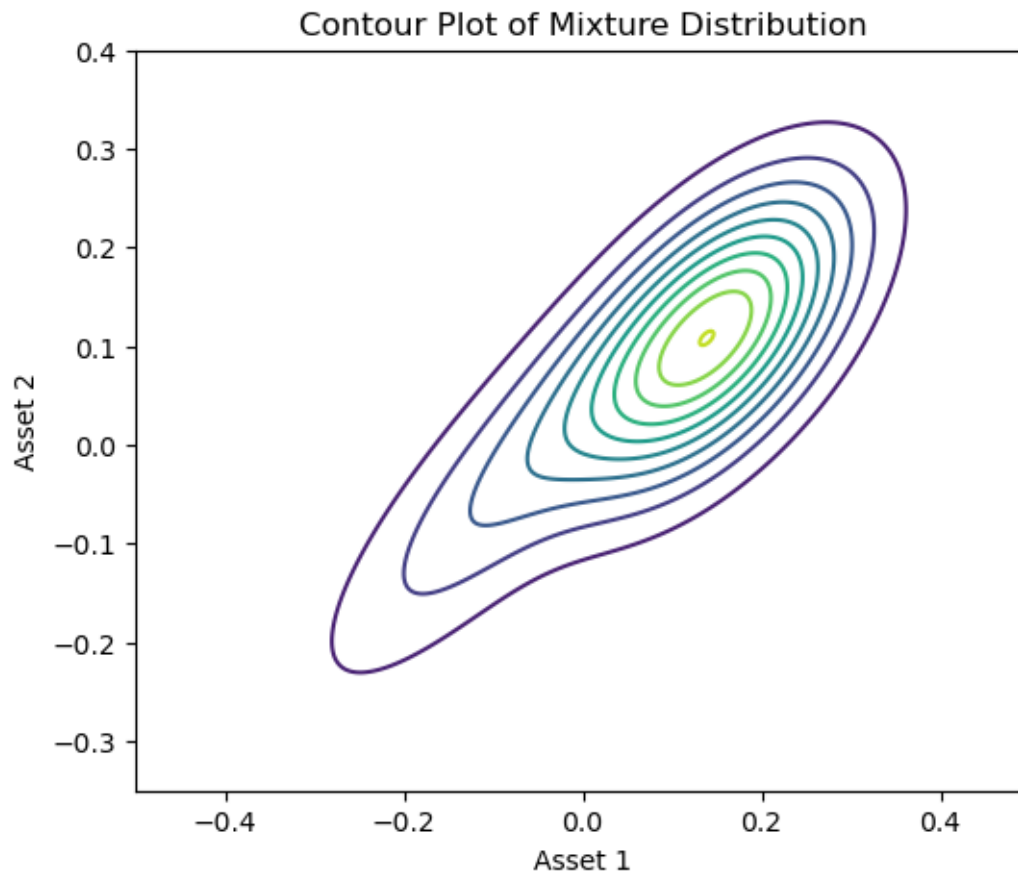
plt.figure(figsize = (6,5))
plt.contour(x1, x2, mixDist, levels = 10, cmap = 'viridis')
plt.xlabel("Asset 1")
plt.ylabel("Asset 2")
plt.title("Contour Plot of Mixture Distribution")
plt.show

```

```

[8]: <function matplotlib.pyplot.show(close=None, block=None)>

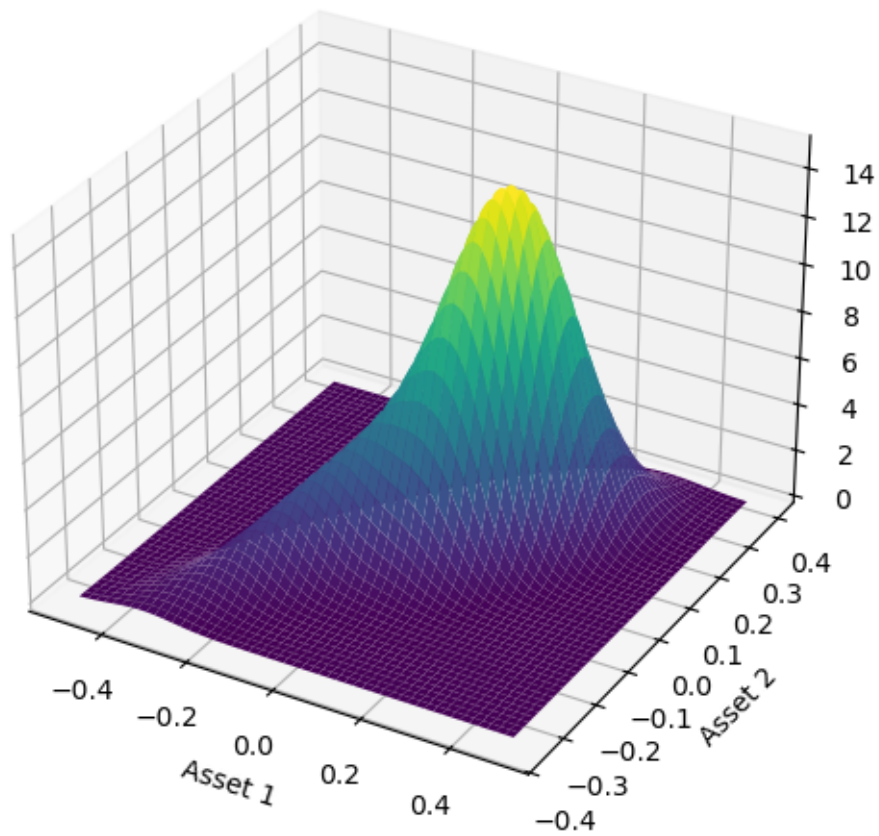
```



```
[10]: # 3D Plot of Mixture Distribution

fig1 = plt.figure(figsize = (8,6))
ax = fig1.add_subplot(111, projection = "3d")
ax.plot_surface(x1, x2, mixDist, cmap = "viridis")
ax.set_xlabel("Asset 1")
ax.set_ylabel("Asset 2")
ax.set_zlabel("Density")
ax.set_title("3D Plot of Mixture Distribution")
plt.show()
```

3D Plot of Mixture Distribution



```
[129]: # An interactive 3D Plot of Mixture Distribution using Plotly
# Allows you to rotate, zoom in and get a good view of the distribution

fig = go.Figure(data=[go.Surface(z=mixDist, x=x1, y=x2, colorscale='viridis')])

fig.update_layout(
    scene=dict(
        xaxis_title="Asset 1",
        yaxis_title="Asset 2",
        zaxis_title="Density"
    ),
    title="3D Plot of Mixture Distribution",
    width=800,
    height=600
)

fig.show()
```

1.6 A Description of the Mixture Distribution

The distribution is a mixture of two bivariate normal distributions representing two market regimes. Regime 1 accounts for 70% of the total distribution, while Regime 2 accounts for the remaining 30%.

Shape:

In most cases, mixture distributions may be bimodal and characterized by two or more peaks in the probability density plot. However, in this case, the distribution appears to be unimodal rather than bimodal. The two peaks merge into one, forming a single peak, as the means of the two underlying distributions are relatively close to each other (0.15 and 0.11). This indicates that the underlying distributions are not well-separated.

Spread and Symmetry:

The distribution has a relatively small spread, which can be attributed to the fact that the covariances between the returns of the two assets are not significantly large. Additionally, the distribution exhibits asymmetry and appears to be skewed. This is expected, as we assign a larger portion of the distribution weight to Regime 1.

1.7 Constructing a Univariate Distribution of Portfolio Returns from the Mixture Distribution

A univariate distribution can be derived from the mixture distribution by fixing a portfolio. This involves choosing a specific set of asset weights in the portfolio, effectively locking in the proportion of each asset within the investment. This method is essential for the following reasons:

- By fixing a portfolio, you effectively “collapse” the mixture distribution into a single, univariate distribution that represents the potential returns of that specific portfolio.
- The resulting distribution is still a mixture of two normal distributions but now applies to portfolio returns instead of two individual asset returns, reducing the distribution to just one variable.
- Fixing a portfolio simplifies the calculation of the expected return by taking a weighted average of the expected returns of each individual asset, where the weights correspond to the portfolio allocation percentages.
- If the individual asset returns are modeled by a mixture distribution, the portfolio return will also follow a mixture distribution, with the weights adjusted based on the portfolio allocation.

Now, suppose we fix a portfolio with asset returns; $x_1 = 0.6$ and $x_2 = 0.4$. We shall compute the univariate distribution of this portfolio. This gives the theoretical portfolio return univariate distribution of the mixture distribution. We will compute and plot the PDF, and determine some descriptive statistics of the distribution.

```
[61]: # Function for Univariate Distribution from mixture distribution
# Here, we apply the function. This gives us an alternative method to compute
# distributions

def uniDistFunc(x, portfolio, weights, means, covariances):
    univarPDF = sum(w * norm.pdf(x, loc=np.dot(portfolio, m), scale=np.
#sqrt(portfolio @ c @ portfolio.T))
```

```

        for w, m, c in zip(weights, means, covariances))
    return univarPDF

```

```

[63]: # Parameters
portfolio = np.array([0.6, 0.4]) # Portfolio allocation
weights = wgts # Mixture weights
means = np.array([mu1, mu2]) # Mean vectors
covariances = np.array([cov1, cov2]) # Covariance matrices
x = np.linspace(-0.45, 0.45, 100) # Random variable range

# Compute the univariate distribution
univar_pdf = uniDistFunc(x, portfolio, weights, means, covariances)

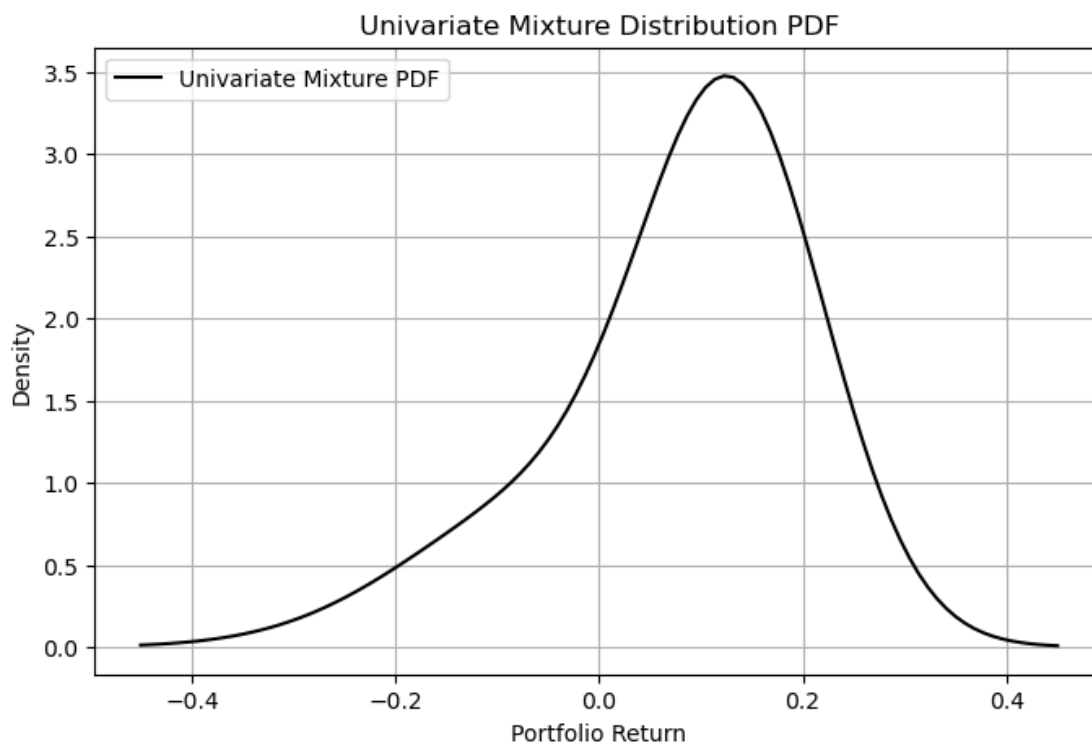
```

1.8 Visualizing the Univariate Distribution

```

[66]: # Plot the PDF
plt.figure(figsize=(8, 5))
plt.plot(x, univar_pdf, label="Univariate Mixture PDF", color='black')
plt.xlabel("Portfolio Return")
plt.ylabel("Density")
plt.title("Univariate Mixture Distribution PDF")
plt.legend()
plt.grid(True)
plt.show()

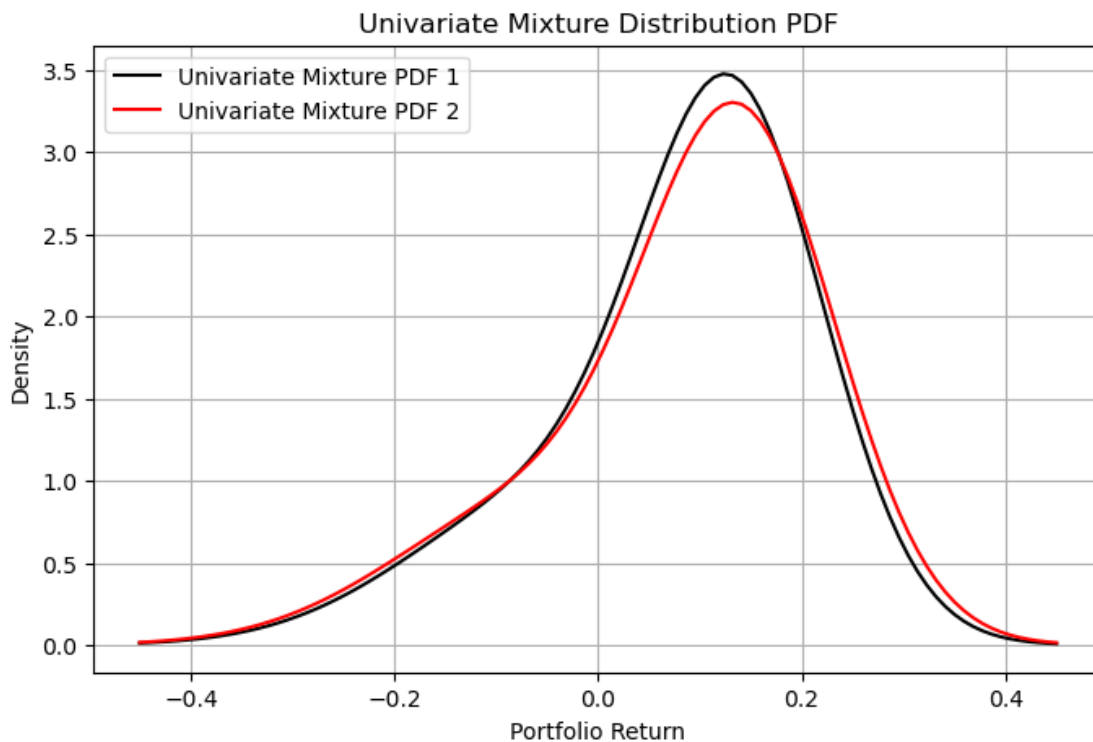
```



Now, suppose we change the weights of the assets in the portfolio to: $x_1 = 0.8$ and $x_2 = 0.2$. The goal is to show changing the weights of the assets in the portfolio will change the univariate portfolio return distribution that can be constructed from the mixture distribution.

```
[73]: portfolio2 = np.array([0.8, 0.2]) # Portfolio allocation
# Compute the univariate distribution
univar_pdf2 = uniDistFunc(x, portfolio2, weights, means, covariances)

# Plot the PDF
plt.figure(figsize=(8, 5))
plt.plot(x, univar_pdf, label="Univariate Mixture PDF 1", color='black')
plt.plot(x, univar_pdf2, label="Univariate Mixture PDF 2", color='red')
plt.xlabel("Portfolio Return")
plt.ylabel("Density")
plt.title("Univariate Mixture Distribution PDF")
plt.legend()
plt.grid(True)
plt.show()
```



1.9 Monte Carlo Simulation of Portfolio Returns

We now simulate 500 portfolio returns from the mixture distribution using the Monte Carlo method. Each simulated return is drawn from the mixture distribution and captures the stochastic nature of the underlying market regimes.

Previously, we derived a theoretical univariate distribution by fixing a portfolio and analytically computing its return distribution from the mixture model. Now, with Monte Carlo simulation, we generate synthetic bivariate asset returns directly from the mixture distribution without fixing a portfolio upfront. After generating the simulated returns, we then fix a portfolio and then map these bivariate returns to portfolio returns by applying a linear combination based on the fixed portfolio weights. This results in an empirical distribution of portfolio returns.

However, the empirical distribution obtained via Monte Carlo is not necessarily the same as the theoretical univariate distribution derived analytically. To analyze potential differences, we will compare the two distributions using histograms and probability plots. This comparison will help assess whether the empirical distribution aligns well with the theoretical distribution, reinforcing confidence in using Monte Carlo simulations for future financial modeling tasks.

```
[107]: # Simulating 500 returns from bivariate mixture distribution

def monteCarloReturns(nSimulations, weights, means, covariances):
    choices = np.random.choice(a = len(weights), size = nSimulations, p =
    weights)
    samples = np.array([multivariate_normal.rvs(mean = means[k], cov =
    covariances[k])
                        for k in choices])
    return samples

# Parameters
nSimulations = 500
portfolio = np.array([0.6, 0.4]) # Portfolio allocation
weights = wgts # Mixture weights
means = np.array([mu1, mu2]) # Mean vectors
covariances = np.array([cov1, cov2]) # Covariance matrices

simulated_returns = monteCarloReturns(nSimulations, weights, means, covariances)
print(simulated_returns.shape)
print(simulated_returns[:5])

(500, 2)
[[0.09225303 0.13326922]
 [0.19009491 0.17635084]
 [0.2999418  0.14872911]
 [0.24429571 0.25466876]
 [0.19148304 0.26249412]]

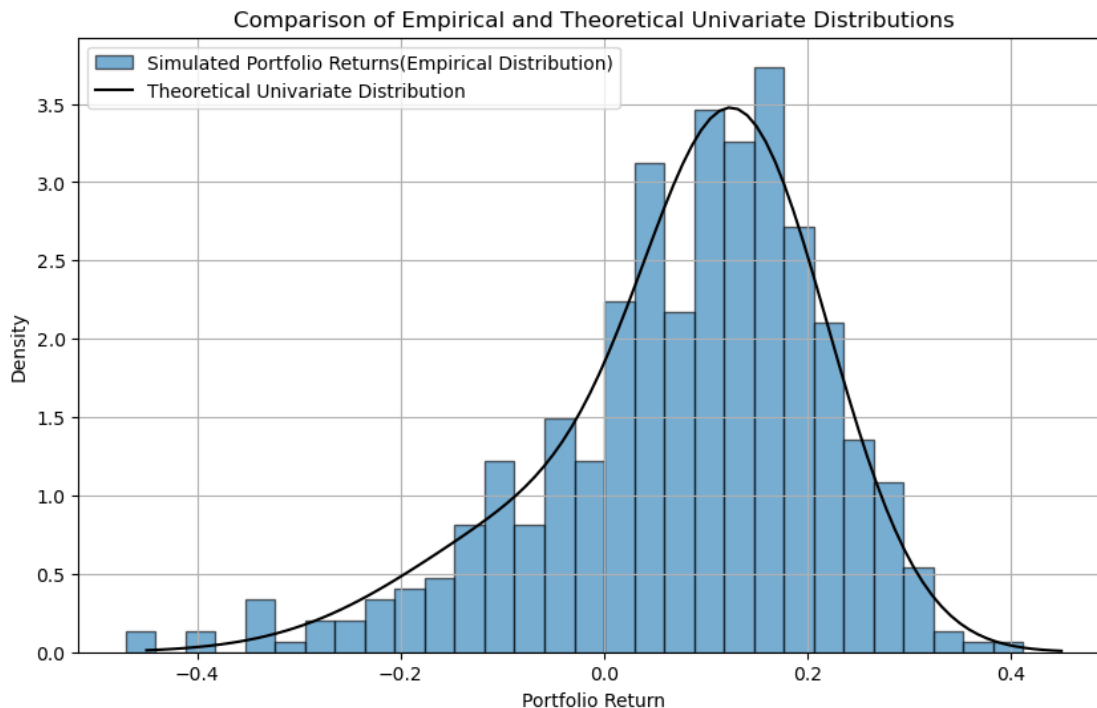
[109]: # Computing the Portfolio Returns
```

```
portfolio_returns = np.dot(simulated_returns, portfolio)
print(portReturns.shape)
print(portReturns[:5])
```

```
(500,)
[0.07493863 0.20767723 0.2023331  0.23968714 0.10396068]
```

```
[131]: # Histogram of the distribution of the Portfolio Returns against the Computed
        ↳ Univariate Distribution
```

```
plt.figure(figsize = (10,6))
plt.hist(portfolio_returns, bins=30, density=True, alpha=0.6, edgecolor =
        ↳ 'black', label="Simulated Portfolio Returns(Empirical Distribution)")
plt.plot(x, univar_pdf, label="Theoretical Univariate Distribution",
        ↳ color='black')
plt.xlabel("Portfolio Return")
plt.ylabel("Density")
plt.title("Comparison of Empirical and Theoretical Univariate Distributions")
plt.legend()
plt.grid(True)
plt.show()
```



```
[137]: # Compute empirical quantiles
sorted_returns = np.sort(portfolio_returns)
```

```

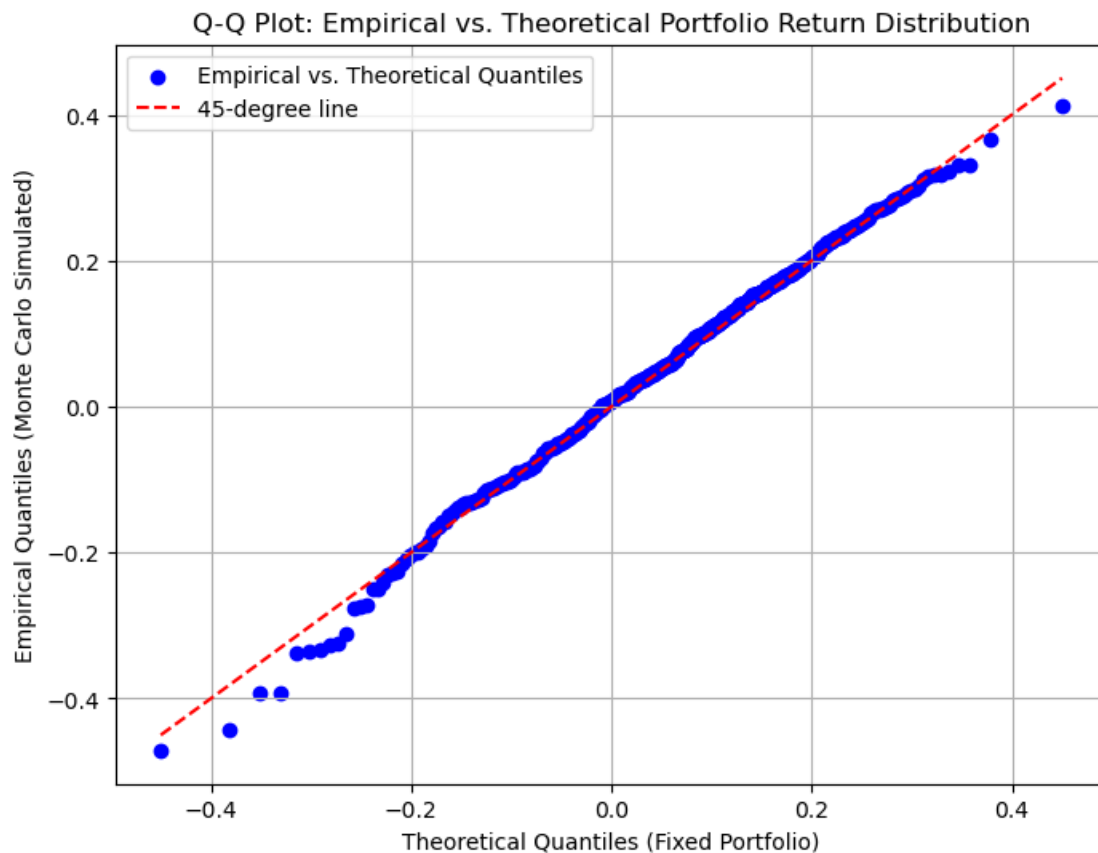
empirical_quantiles = np.linspace(0, 1, len(sorted_returns))

# Compute theoretical quantiles from the univariate mixture distribution
theoretical_cdf = np.cumsum(univar_pdf) / np.sum(univar_pdf) # Normalized CDF
theoretical_quantiles = np.interp(empirical_quantiles, theoretical_cdf, x) #
    ↳Get theoretical quantiles

# Plot Q-Q Plot
plt.figure(figsize=(8, 6))
plt.scatter(theoretical_quantiles, sorted_returns, label="Empirical vs.
    ↳Theoretical Quantiles", color="blue")
plt.plot(theoretical_quantiles, theoretical_quantiles, color="red",
    ↳linestyle="dashed", label="45-degree line")

plt.xlabel("Theoretical Quantiles (Fixed Portfolio)")
plt.ylabel("Empirical Quantiles (Monte Carlo Simulated)")
plt.title("Q-Q Plot: Empirical vs. Theoretical Portfolio Return Distribution")
plt.legend()
plt.grid(True)
plt.show()

```



From the Q-Q plot above, the Monte Carlo simulation performs well enough when compared to the theoretical univariate distribution.

1.10 References

1. Everitt, B. (2013). Finite mixture distributions. Springer Netherlands.
<https://books.google.com/books?id=0ZQICQAAQBAJ>
2. Luxenberg, Eric and Boyd, Stephen, (2022), Portfolio Construction with Gaussian Mixture Returns and Exponential Utility via Convex Optimization, Papers, arXiv.org.
3. Frey, R. (2024). AMS512:Portfolio Theory, Lecture 01: Mixture Distributions. Stony Brook University.

[]: